TCP

(Transmission Control Protocol)

Computer **Networks**

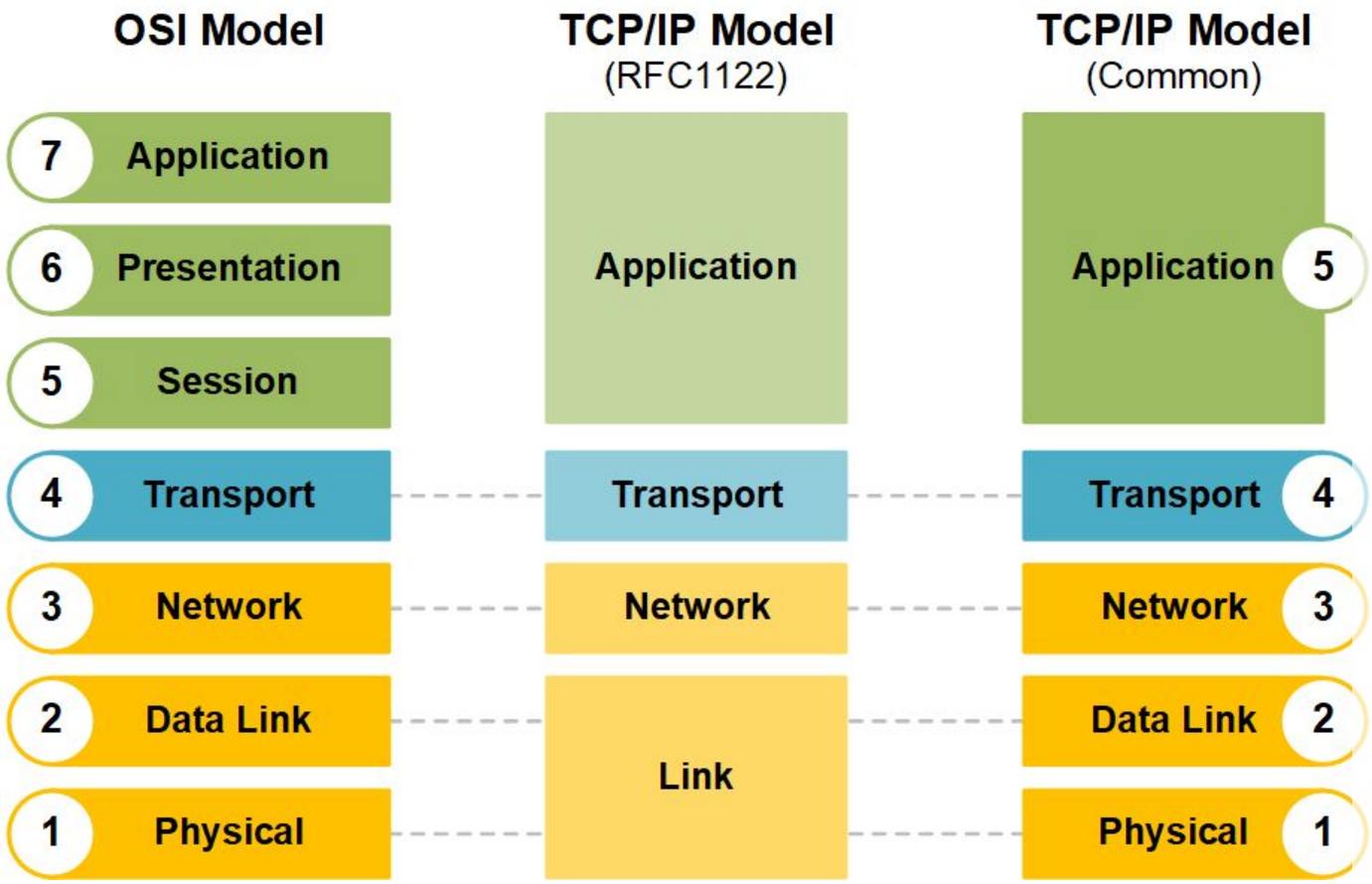


UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

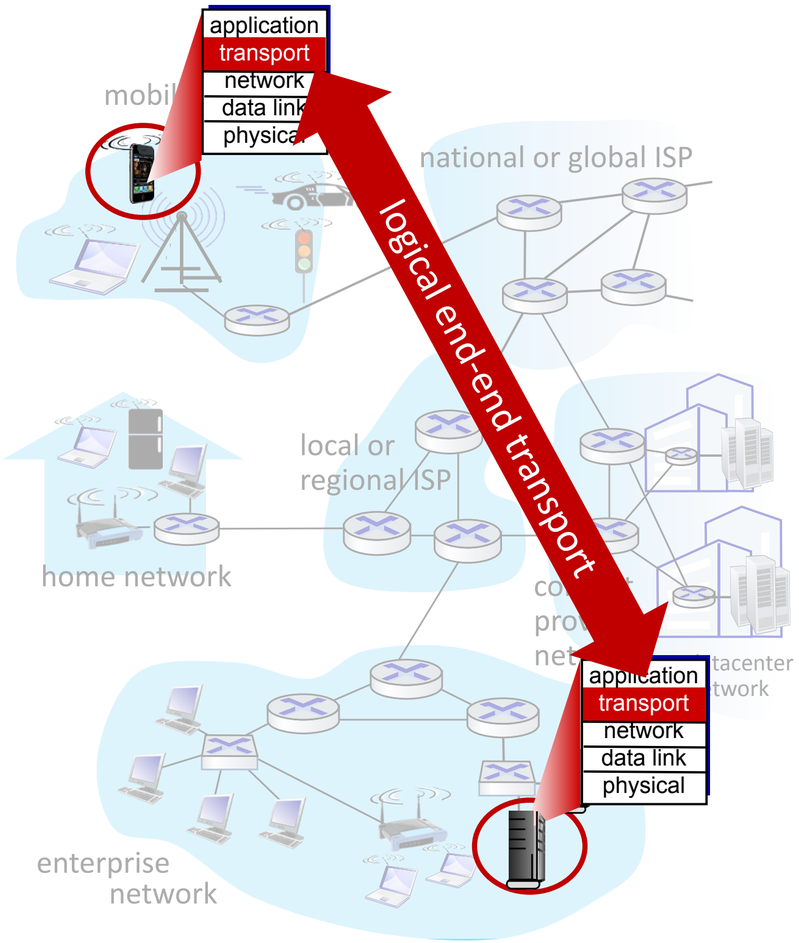
CS4054 - 2026-1
PROF.: garias@utec.edu.pe
SRC: KUROSE – COMPUTER NETWORKS BOOK
PETERSON AND DAVIE – COMPUTER NETWORKS BOOK

Transport Layer

It acts as the liaison between the **Application layer** and the lower **network layers.**



Src. NetworkAcademy.io

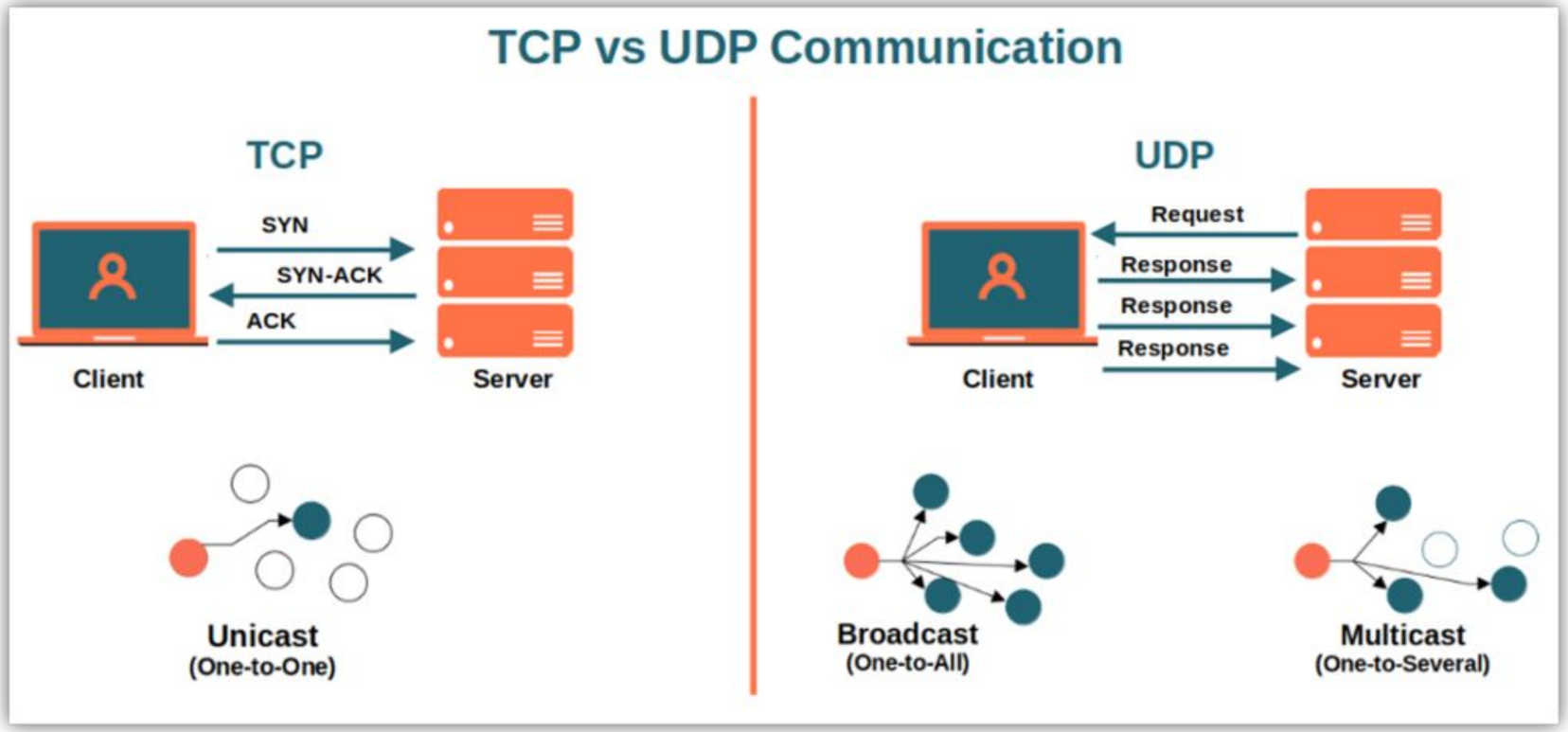


garias@utec.edu.pe

Transport Layer

There are two primary ways to move data across the network:

1. **TCP**: **Connection-oriented**. Focuses on **reliability**.
2. **UDP**: **Connectionless**. Focuses on **speed**.



Src. Cheap Ssl Security.

garias@utec.edu.pe

TCP vs UDP

Reliability vs. Speed

Feature	TCP	UDP
Connection	Handshake required (3-way)	No handshake (Connectionless)
Delivery	Guaranteed & In-order	Best-effort (No guarantee)
Error Recovery	Retransmits lost packets	No retransmission
Overhead	Heavy header (20 bytes)	Lightweight header (8 bytes)

garias@utec.edu.pe

TCP

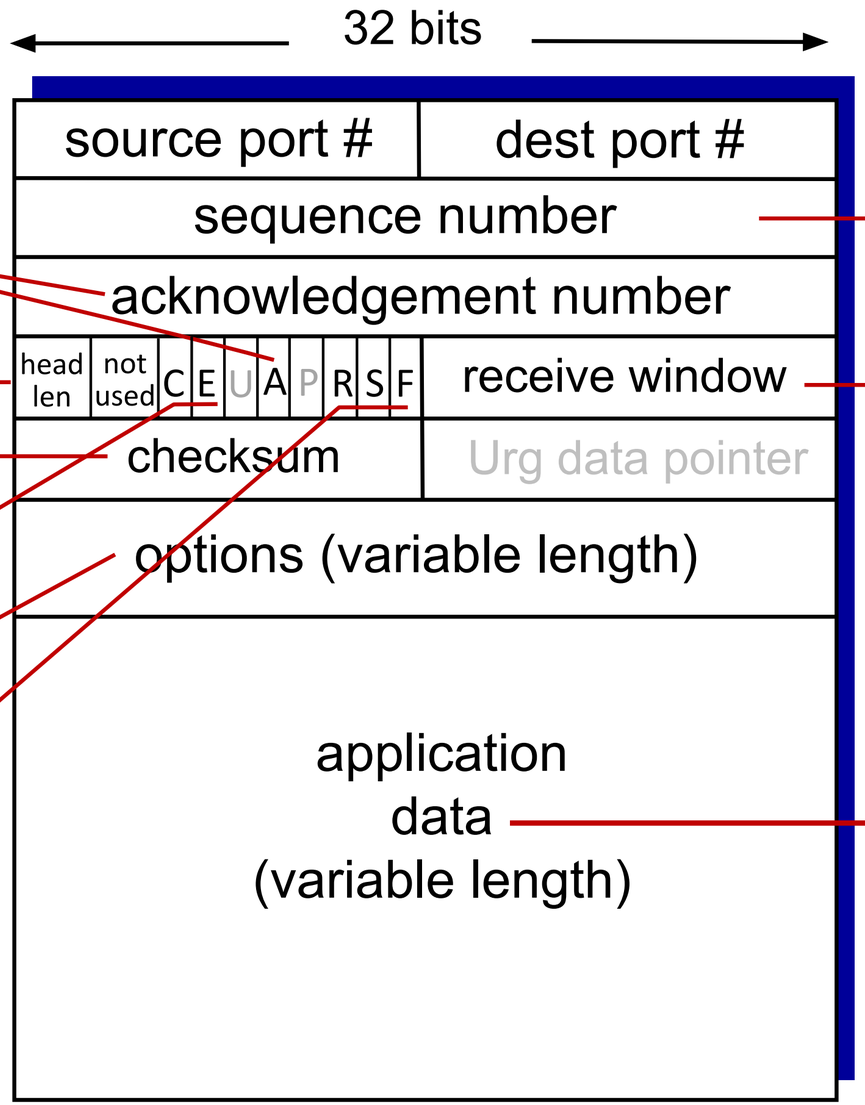
Transmission Control Protocol

RFCs: 793,1122, 2018, 5681, 7323

- point-to-point:
  - one sender, one receiver
- reliable, in-order *byte steam:*
  - no “message boundaries"
- full duplex data:
  - bi-directional data flow in same connection
  - MSS: maximum segment size
- cumulative ACKs
- pipelining:
  - TCP congestion and flow control set window size
- connection-oriented:
  - handshaking (exchange of control messages) initializes sender, receiver state before data exchange
- flow controlled:
  - sender will not overwhelm receiver

garias@utec.edu.pe

TCP



ACK: seq # of next expected byte; A bit: this is an ACK

length (of TCP header)

Internet checksum

C, E: congestion notification

TCP options

RST, SYN, FIN: connection management

segment seq #: counting bytes of data into bytestream (not segments!)

flow control: # bytes receiver willing to accept

data sent by application into TCP socket

garias@utec.edu.pe

TCP

Sequence numbers:

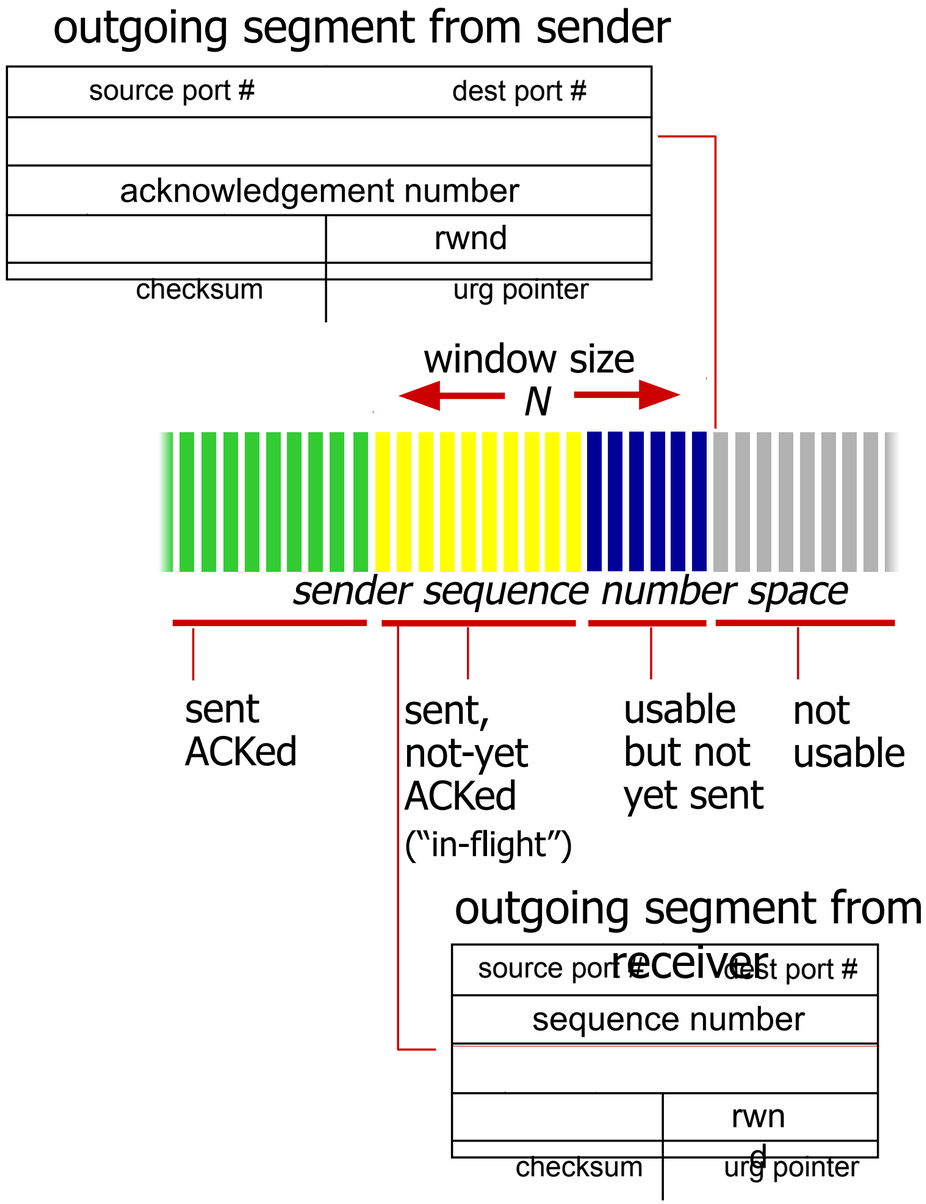
- byte stream “number” of first byte in segment’s data

Acknowledgements:

- seq # of next byte expected from other side
- cumulative ACK

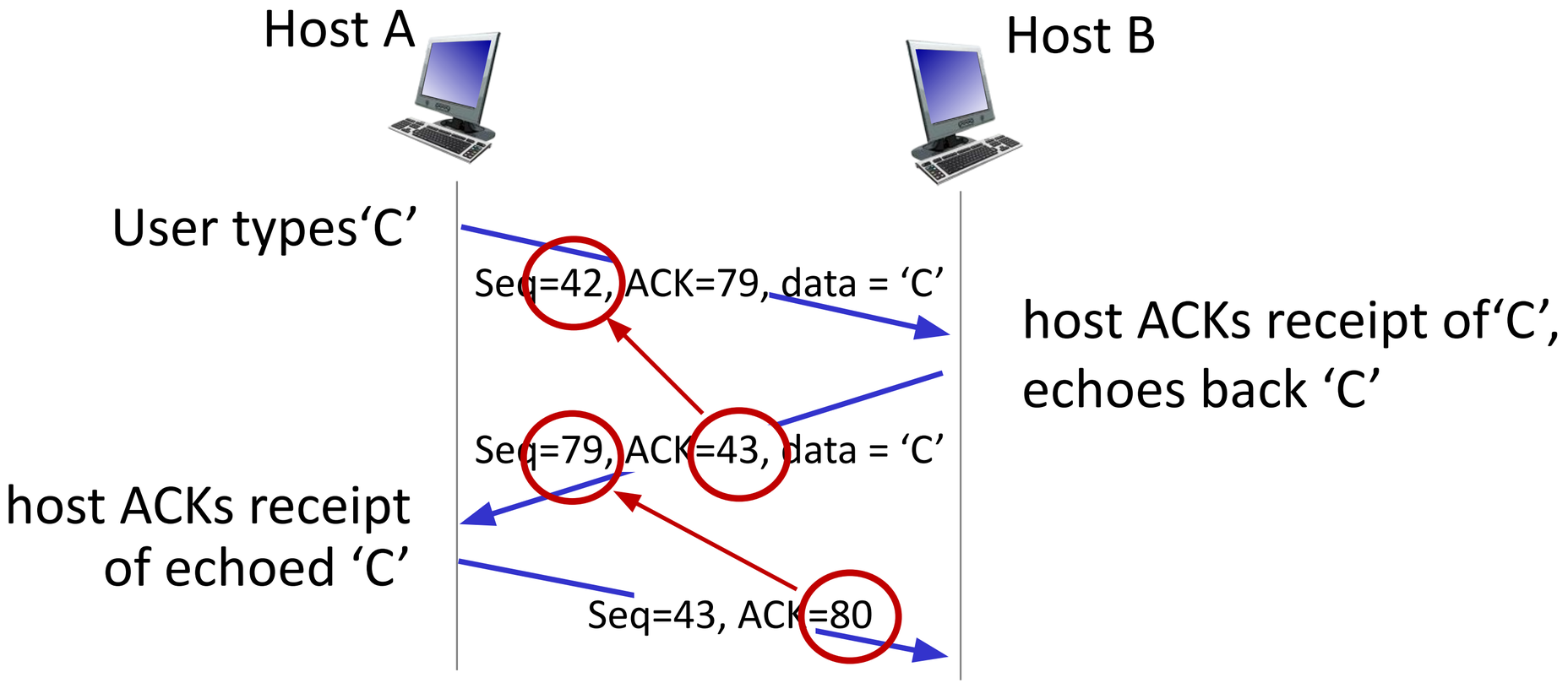
Q: how receiver handles out-of-order segments

- *A*: TCP spec doesn’t say, - up to implementor



garias@utec.edu.pe

TCP



simple telnet scenario

garias@utec.edu.pe

RTT Calculation and Estimation Methodology

To characterize network latency dynamics, the round-trip time for each segment is evaluated through two metrics: the instantaneous measurement (**SampleRTT**) and a smoothed historical average (**EstimatedRTT**).

➔ **SampleRTT:** Defined as the elapsed time between the transmission of a given TCP segment (starting with the HTTP POST request) and the reception of its corresponding acknowledgment (ACK):

$$\text{SampleRTT} = t_{\text{ACK}} - t_{\text{Sent}}$$

garias@utec.edu.pe

RTT Calculation and Estimation Methodology

➔ **EstimatedRTT:** Because instantaneous network conditions are subject to transient fluctuations and jitter, TCP relies on an Exponentially Weighted Moving Average (EWMA) to estimate the expected RTT. For the initial segment, the baseline is set directly to the first measured value According to RFC 6298.

Where **alpha=0.125**, placing an **87.5%** weight on historical estimates and a **12.5%** weight on the most recent sample. This smoothing dampens short-term spikes while ensuring the estimator dynamically adapts to persistent latency trends.

$$\text{EstimatedRTT}_n = (1 - \alpha) \cdot \text{EstimatedRTT}_{n-1} + \alpha \cdot \text{SampleRTT}_n$$

garias@utec.edu.pe

RTT Calculation and Estimation Methodology

Where **alpha=0.125**, placing an **87.5%** weight on historical estimates and a **12.5%** weight on the most recent sample. This smoothing dampens short-term spikes while ensuring the estimator dynamically adapts to persistent latency trends.

$$\text{EstimatedRTT}_n = 0.875 \cdot \text{EstimatedRTT}_{n-1} + 0.125 \cdot \text{SampleRTT}_n$$

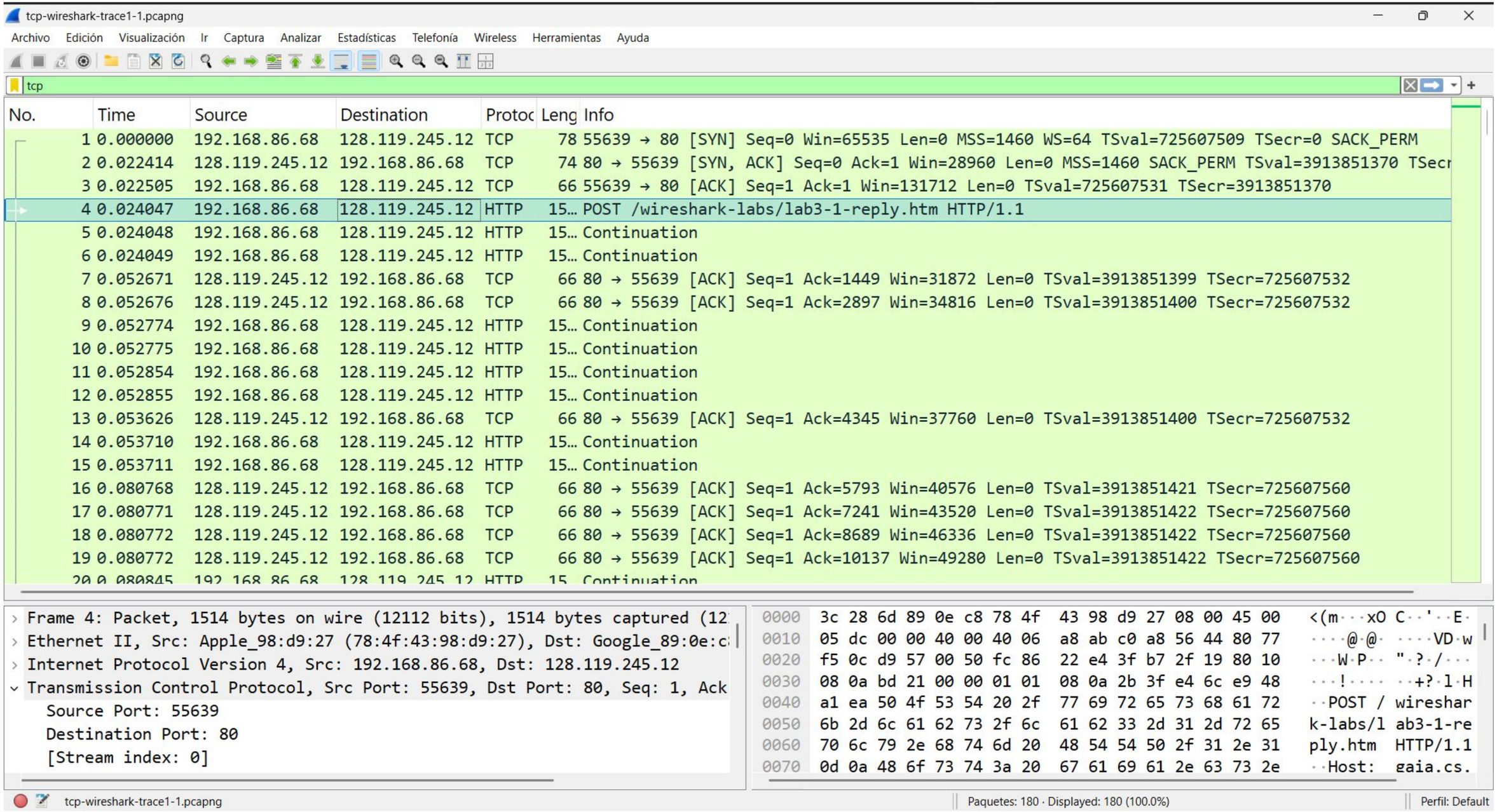
garias@utec.edu.pe

RTT Calculation and Estimation Methodology

Where **alpha=0.125**, placing an **87.5%** weight on historical estimates and a **12.5%** weight on the most recent sample. This smoothing dampens short-term spikes while ensuring the estimator dynamically adapts to persistent latency trends.

$$\text{EstimatedRTT}_n = 0.875 \cdot \text{EstimatedRTT}_{n-1} + 0.125 \cdot \text{SampleRTT}_n$$

garias@utec.edu.pe



garias@utec.edu.pe

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

TCP

garias@utec.edu.pe